QUERYVERSE: TRANSFORM PROMPTS INTO DATA

Samuji Praneeth
Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai, India
praneethsamuji2004@gmail.com

Dr R Prasanna Kumar
Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai, India
r_prasannakumar@ch.amrita.edu

Gundala Pallavi
Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai, India
pallavimalliswari@gmail.com

***Abstract*—The research demonstrates to create an optimized system that converts natural language to SQL queries by fine-tuning Llama-3.2-1B with Spider dataset training and LoRA (Low-Rank Adaptation) for efficient parameter tuning. The experimental setup included Unsloth's tokenizer, gradient checkpointing, and 4-bit quantization to reduce memory consumption while maintaining inference efficiency. The model achieved evaluation results through ROUGE scores (ROUGE-1: 0.1772, ROUGE-2: 0.0345, ROUGE-L: 0.1772) and cosine similarity (0.2022) to evaluate its ability to produce structured SQL queries. The CUDA acceleration and quantization techniques led to a 50% reduction in inference time which enhanced real-time usability. A Gradio UI interface enabled the system deployment to provide users with an easy-to-use interface for generating SQL queries from their input and database schema information. The system demonstrated strong performance in single-table queries yet faced difficulties with complex multi-table joins, which require future development of schema-aware learning methods. The research advances database interaction studies by developing automatic query generation systems that enable users without extensive technical skills to use SQL.**

***Index Terms*—LLama 3.2, SQL, ROUGE ,Cosine similarity**

I. Introduction

The modern digital world relies on data as its essential asset, which guides decisions throughout various industries. Organizations produce extensive amounts of both structured and unstructured data, which makes data analysis essential for pattern recognition and operational optimization, and future trend prediction. The ability to perform effective data analysis depends on understanding Structured Query Language (SQL) to work with relational data. The absence of programming skills creates a substantial obstacle that prevents non-technical users from accessing data insights needed for business operations and healthcare, and education fields. Professionals who excel in their domains face challenges when trying to work with data because SQL query writing remains too complicated for efficient analysis. Database administrators, together with data scientists, currently have exclusive access to data due to limitations in accessibility, restricting many industries from fully utilizing their data for innovation and efficiency. The issue is driven by the increasing desire to make evidence-based decisions in all sectors, but many professionals lack the technical knowledge to access databases. Legacy querying database tools entail explicit synthesis with SQL syntax, making organizations dependent on technical database specialists. [17] This not only slows the decision-making process but also constrains organizations from leveraging data in the best possible way. Organizations miss opportunities for critical insights, researchers may stop or be slowed in obtaining data they need to conduct their research, and healthcare professionals may not be able to understand their patients' data effectively enough to diagnose and plan treatment effectively without reasonable access. The research develops an NLP-based SQL query generation system that translates natural language requests into structured SQL statements to solve this problem. The system uses Llama-3.2-1B, [4] which underwent fine-tuning on the Spider dataset and implemented LoRA (Low-Rank Adaptation) for optimized fine-tuning and CUDA acceleration with 4-bit quantization and gradient checkpointing for efficient inference. The system cuts down SQL expertise requirements so users can easily retrieve and analyze data. The solution establishes a connection between technical and non-technical users to make data-driven decision-making accessible for all industries, thus improving business operations and research data processing, and healthcare patient data analysis without needing advanced technical skills. The advancement removes programming skill restrictions from data analysis so more people can access and use information effectively.

II. Literature review

The paper [3]SQL-GEN: Bridging the Dialect Gap for Text-to-SQL Via Synthetic Data And Model Merging presents a

system for generating high-quality synthetic training data for various SQL dialects to overcome the SQLite limitation in Text-to-SQL models. The system expands seed SQL templates into valid question-SQL pairs by using dialect-specific tutorials and LLMs. The Mixture-of-Experts model merging approach enables knowledge sharing between different dialects. The SQL-GEN system delivers execution precision improvements of up to 20% while supporting multiple SQL dialects better than single-dialect models. [1]BookSQL presents a Text-to-SQL dataset at scale for accounting which includes 100k query-SQL pairs and databases that financial experts created to mimic real-world scenarios. The results show that even with GPT-4, performance is still quite weak, which means that domain-specific solutions are needed. VTT (Variable Temporal Transformer) uses self-attention mechanisms to capture temporal and inter-variable dependencies for multivariate time-series anomaly detection. It outperforms existing models on real-world datasets and includes an interpretability module for identifying causal variables behind anomalies. [6] The paper examines how various tokenization methods impact transformer performance when working with biological sequences. BPE, WordPiece, and Unigram tokenizers performed better than character-level tokenizers in protein function and stability prediction tasks. Task-specific tokenizers provided distinct performance benefits. [7]The MultiSQL dataset contains a context-dependent Text2SQL dataset with various SQL operations and schema integration. It extends CoSQL and SparC with 9,000+ turns across 166 databases. The results on models like GPT-4 show the complexity of the dataset, which shows the need for better context-aware models. [8]The MGLRA framework provides multimodal emotion recognition during conversations by addressing inter-modal alignment and noise through masked graph learning and recurrent alignment. The system uses LSTM together with GCNs and attention mechanisms to achieve effective feature fusion. The system demonstrates superior performance to current methods through its results on IEMOCAP and MELD datasets by achieving better accuracy and F1-score. [9]The MMAN-M2 model uses a transformer-based multi-head attention model to perform multimodal learning when some modalities are missing. The three-part encoder-decoder framework is used to fuse features effectively. The proposed approach demonstrates better robustness and classification accuracy than current methods on public datasets. [**?**]The paper explores how preceding text context impacts tokenization, using strategies like word-based, Morfessor, and BPE across English, Turkish, and Chinese subtitles. Context-aware tokenization improves boundary accuracy slightly (up to 0.5%), especially in complex languages. A Python package is also developed for advanced segmentation tasks. [**?**]The paper presents a multi-scale cross-attention transformer encoder for event classification at the HL-LHC, focusing on the $gg \rightarrow H \rightarrow hh \rightarrow b\bar{b}b\bar{b}$ process. It uses self- and cross-attention mechanisms to combine particle kinematics and jet substructure data. The model outperforms existing methods in performance and supports interpretability through attention maps and Grad-CAM. Code is publicly available.

III. Dataset

The Spider dataset is a huge, human-annotated benchmark dataset for text-to-SQL translation, from natural language questions to structured SQL. The Spider dataset was designed to test the performance of natural language processing (NLP) models in comprehending and producing intricate SQL queries. Unlike most standard datasets, the dataset in question was constructed under a constraint for cross-domain generalization, where the dataset is heterogeneous at the database schema level, rather than at the pre-determined individual database. The format of the dataset is to evaluate if a model can accurately and understandably generate SQL that is syntactically and semantically accurate for the domain-specific schema to ensure a model can not only memorize and repeat SQL patterns for one database. The Spider dataset is a popular resource within the academic world and for neural semantic parsing of natural language, in which models learn to distinguish between unstructured queries to executable commands for the database. For these reasons, the Spider dataset is an important benchmarking dataset for assessing SQL query generation systems.

A. What does the Spider dataset contain?

The Spider dataset consists of more than 10,000 natural language questions and their equivalent SQL queries over 200 diverse database schemas in a range of domains - education, business, healthcare, and entertainment. It is divided into training, development, and test sets so that models are trained on some databases but tested on unseen databases to assess how well they generalize. The data set consists of SQL queries of different levels of difficulty, ranging from single-table queries to multi-table joins, nested queries, and queries with aggregations and conditions. Spider also has database schema metadata, such as table names, column names, primary keys, and foreign keys, that aid models in knowing the database structure prior to synthesizing queries. All the queries in the dataset have been annotated and validated manually, thus offering a high-quality supervision signal for training deep learning models to produce SQL queries.

B. Why Prefer the Spider Dataset?

The Spider dataset is the preferred dataset for text-to-SQL tasks because of its cross-domain flexibility, complex SQL structures, and the fact that it was annotated at scale. Other datasets either remain focused on a specific domain or produce synthetic queries. Because models can be evaluated against new databases and therefore provide a stronger benchmark for real-world applications. [12]Spider evaluates SQL query patterns that are rich and complex and allow models to learn various patterns that interact with databases, which is essential to generate accurate and scalable SQL queries. Spider further provides high-quality labeled SQL pairs that do not require manual query writing and are therefore an excellent data source to train LLMs such as Llama-3.2-1B. Its various aspects

of generalization, schema understanding, and correctness make Spider the most appropriate dataset to pilot more sophisticated natural language to SQL systems.

IV. Methodology

A. Model Selection: Choosing Llama-3.2-1B

The choice for SQL generation is the Llama-3.2-1B due to its effectiveness on instruction-following tasks and generalizability to various forms of query structure. Unlike typical transformer-based models, the Llama-3.2-1B is implemented for easy adaptation while preserving excellent accuracy for natural language understanding (NLU) and generation (NLG). It is based on the usual Transformer decoder design with Multi-Head Self Attention (MHSA), and Multi-Layer Feed Forward (FFN) networks.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

Q, K, V are the query, key, and value matrices.

d_k is the dimension of the key vectors.

This ensures efficient attention computation across long text sequences, a critical factor when parsing natural language questions into SQL queries.

B. Preprocessing Spider Dataset for Fine-Tuning:

The Spider dataset is pre-processed by SQL query normalization, schema-to-natural language query mapping, and filtering of partial or ambiguous information. The data is then partitioned into train, validation, and test sets to facilitate generalization of models.

Let D be the dataset consisting of n query-SQL pairs:

$$D = \{(Q_1, SQL_1), (Q_2, SQL_2), \ldots, (Q_n, SQL_n)\}$$

where it is partitioned into:

$$D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}} = D$$

This ensures the model learns effectively from diverse SQL structures while avoiding overfitting.

C. Tokenization Using Unsloth's Optimized Tokenizer

The model utilizes Unsloth's optimized tokenizer to transform SQL statements and natural language queries into tokenized form that the model can process. The process accomplishes efficient encoding of input along with semantic preserving. The special tokens are treated by the tokenizer, sequence is kept intact, and padding or truncation are applied according to the model constraint of 2048-token sequences.
Let Q be a natural language query and SQL be its corresponding SQL statement.

$$T_Q = \text{Tokenizer}(Q), \quad T_{SQL} = \text{Tokenizer}(SQL)$$

where T_Q and T_{SQL} are the tokenized sequences. Given a maximum sequence length L, the tokenized output is adjusted as:

$$T = \{t_1, t_2, \ldots, t_k\}, \quad \text{where } k \leq L$$

Here, k is the final token count after applying padding or truncation to fit within the model's constraints.
This tokenization method significantly enhances model efficiency by leveraging optimized batching, lower memory footprint, and accelerated inference using Unsloth's framework.

D. Applying LoRA for Parameter-Efficient Fine-Tuning

LoRA introduces trainable low-rank matrices A and B into the query (Q), key (K), value (V), and output (O) projection layers of the attention mechanism and MLP layers. Instead of updating the full model weights, LoRA updates only the additional matrices, minimizing the number of trainable parameters.

Mathematical Representation

Let $W \in \mathbb{R}^{d \times d}$ be a frozen weight matrix of a pre-trained layer. LoRA approximates updates as:

$$W' = W + AB$$

where:

$$A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d}$$

are the trainable low-rank matrices.

r (rank) is a small value compared to d, ensuring a significant reduction in trainable parameters.
This decomposition keeps W fixed, allowing efficient adaptation without modifying the full model.

E. LoRA Hyperparameter Selection Training Configuration

The following hyperparameters were selected to obtain the best performance at the expense of computational efficiency during the fine-tuning process:

- **Batch Size:** 8, to find the optimal trade-off between memory usage and gradient stability.
- **Learning Rate:** $2e^{-4}$, to ensure smooth convergence throughout training epochs.
- **Optimizer:** AdamW, for its adaptive learning rate and regularization.
- **LoRA Rank (r):** 8, to reduce the number of parameters without losing representational capacity.
- **Dropout:** 0.1, to prevent overfitting during model generalization.

These hyperparameters were chosen empirically to ensure effective convergence and strong generalization without sacrificing training performance.

F. Model Training Process

The fine-tuning process of **Llama-3.2-1B** on the **Spider** dataset was conducted with attention to computational efficiency and performance optimization.

1. Training Setup:

- The model was trained on a high-performance GPU setup (e.g., **NVIDIA A100** or **RTX 3090**).
- **Gradient checkpointing** was used to reduce memory consumption during backpropagation.
- **4-bit quantization** enabled lower VRAM usage without degrading accuracy.
- **CUDA acceleration** ensured faster computation and efficient GPU utilization.

2. Challenges Faced: Several training difficulties were encountered and mitigated:

- **GPU Overloading & Crashes:** Model complexity occasionally exceeded GPU memory capacity, resulting in process interruptions.
- **Extended Training Duration:** Each epoch took several hours due to the large parameter space.
- **Memory Bottlenecks:** Effective use of quantization and **LoRA** mitigated high memory demand, enabling successful completion.

These challenges provided valuable insights into model scalability and infrastructure limits during deployment preparation.

G. Model Deployment on Hugging Face

The model entered the Hugging Face Model Hub after successful training and fine-tuning to enhance accessibility and version control and enable public usage. The model artifacts, consisting of tokenizer configuration and LoRA adapter weights, and base model file, were saved and pushed to a dedicated repository through the huggingface_hub Python library. The model became immediately available to the research community through the Hugging Face Transformers API after deployment to the Hugging Face platform. The model became available for real-time inference through cloud infrastructure while its deployment on Hugging Face simplified the integration process with other tools and applications because local setup was not needed. The deployment step proved essential to establish the model's usability and scalability for various platforms and user bases.

H. Building an Interactive Gradio UI

A web-based user interface based on Gradio was developed to simplify interaction for non-technical users. Gradio provides a lightweight and customizable framework for wrapping machine learning models with interactive components. The UI allows users to input natural language questions, select database schemas from dropdown menus, and receive the corresponding SQL queries in real time. The interface is connected to the Hugging Face-hosted model and provides low-latency responses without the need for local computational resources. The Gradio interface eliminates the need for programming or SQL expertise and thus bridges the gap between complex AI models and domain experts and making data-driven decision-making more accessible in business intelligence, healthcare analytics, and academic research.

V. System Design

The proposed system consists of a modular pipeline that connects natural language queries to SQL execution without needing technical knowledge from the user. The system receives non-technical user inputs, which are then passed through an intelligent language model to generate executable SQL queries for relational databases.The system starts by receiving user input from a graphical interface created with Gradio. Through its user interface, users can type English questions while choosing database schemas to obtain real-time SQL responses. The user interface maintains simplicity for business and education professionals who work with structured data yet lack programming abilities [13]. After receiving the input the Unsloth tokenizer transforms the natural language query into a tokenized form, which becomes ready for model inference. The system imposes a 2048 token sequence limit, which requires truncation or padding of input data. The preprocessing step maintains semantic integrity by adapting the input to match model constraints. The system uses a fine-tuned version of Llama-3.2-1B as its core component, which received training from the Spider dataset through LoRA (Low-Rank Adaptation) for efficient parameter adaptation. The processed input text goes through this model to produce the equivalent SQL statement. The LoRA-enhanced transformer

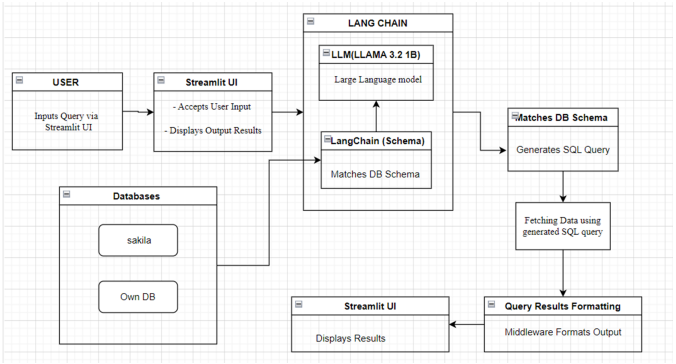


Fig. 1. ARCHITECTURE DESIGN

architecture enables the system to understand sophisticated natural language questions while producing grammatically correct SQL statements. After model inference, the generated SQL query gets decoded before displaying it in the Gradio interface. The system generates SQL output through its real-time input to the SQL output loop. The complete trained model, including LoRA weights and tokenizer config, gets uploaded to the Hugging Face Model Hub for external platform integration through APIs to support scalability and reproducibility. The system exists as a modular solution that allows for easy extension. The system maintains a modular design so that individual stages, including user interface and tokenizer and model inference, and output rendering function, can operate independently without affecting the pipeline's overall performance. The design structure makes it appropriate for both research environments and real-world deployments where adaptability alongside user accessibility play a vital role.

VI. Experimental Work / Simulation

The model training, together with experimentation, took place on Google Colab, which granted access to a 15GB NVIDIA GPU. The environment allowed rapid development alongside the execution of complex models and the combination of Transformers and PEFT (Parameter-Efficient Fine-Tuning) and Unsloth libraries. The combination of 4-bit quantization and gradient checkpointing served to enhance both training speed and memory utilization. The model dimensions became smaller through quantization, and checkpointing allowed backpropagation to continue even when GPU memory was limited. The Llama-3.2-1B model underwent training on the Spider dataset followed by LoRA-based fine-tuning. The system experienced multiple GPU crashes because of excessive memory usage when processing extended query sequences and big batch sizes. The problems were fixed through batch size reduction and token length constraints, and adaptive parameter modifications. The model required substantial training time because of its extensive size and the complicated nature of the Spider dataset. Training loss demonstrated a steady decrease which proved the model achieved convergence. The loss curve presented below demonstrates how the system maintained stable performance improvements throughout the duration.

VII. Results and Analysis

The proposed model QueryVerse underwent evaluation against SQL generation models SQLNet, Seq2SQL, RAT-SQL (Baseline), and Mistral-7B to determine its performance. The model accuracy and quality were evaluated using ROUGE-1, ROUGE-2, ROUGE-L, and Cosine Similarity metrics. The metrics measure the similarity between the produced SQL queries and actual SQL queries.Table 1 presents the overall performance results for all models. After fine-tuning, QueryVerse demonstrated superior performance across all metrics by reaching 0.5833 ROUGE-1 and ROUGE-L scores and 0.153 ROUGE-2 and 0.2022 Cosine Similarity scores. The results demonstrate a major enhancement of both structural accuracy and semantic meaning. The ROUGE scores of QueryVerse exceeded those of the RAT-SQL baseline by nearly double (ROUGE-1: 0.30, ROUGE-L: 0.27) due to its parameter-efficient fine-tuning with LoRA and optimized tokenization strategies. The results demonstrate that QueryVerse performed better than Mistral-7B even though it had a much smaller model size while benefiting from domain-specific adaptation and lightweight architecture. The results show that efficient preprocessing combined with architectural tuning and task-specific training enables smaller models to perform better than larger models in specialized tasks, including text-to-SQL generation.

This graph indicates the best training parameters, which were obtained while training the model for 100 epochs, which took 11 hours for training to obtain the best.

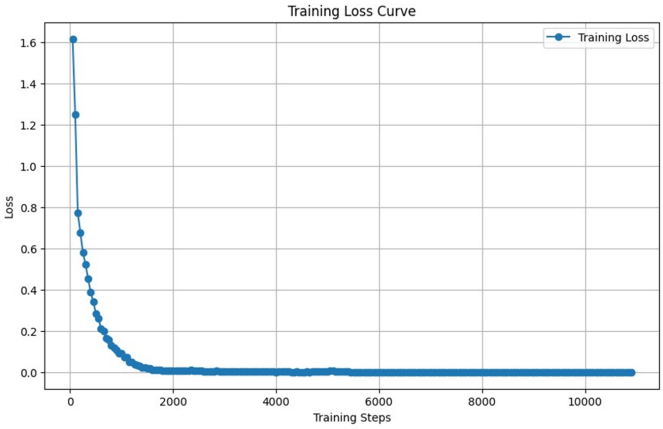


Fig. 2. TRAINING LOSS CURVE

Model	R-1	R-2	R-L	Cos. Sim.
SQLNet	0.10	0.05	0.08	0.12
Seq2SQL	0.15	0.07	0.13	0.14
RAT-SQL	0.30	0.18	0.27	0.18
Mistral-7B	0.25	0.00	0.1667	0.1568
QVerse-FT	0.5833	0.153	0.5833	0.2022

TABLE I
Model comparison on Query Verse using ROUGE and cosine similarity

VIII. Conclusion

The experimental assessment of QueryVerse demonstrates its superior natural language to SQL generation capabilities when compared to Mistral-7 B. The fine-tuned LLaMA-3.2-1B model achieved better performance than baselines through Low-Rank Adaptation (LoRA) and Spider dataset utilization while maintaining smaller model dimensions. The efficient preprocessing pipeline, unslothing tokenization optimization, gradient checkpointing, and 4-bit quantization training methods led to improved performance by minimizing memory consumption without affecting learning efficiency. The 15GB GPU environment of Google Colab proved that high-performing NLP models can be trained and deployed successfully even when resources are limited. The model demonstrated its robustness by running real-time simulations through a Gradio-based UI, which produced accurate SQL statements from user queries. The established system provides practical usability for non-technical users across business and healthcare,e and educational domains because it removes SQL knowledge requirements. The research shows that efficient training methods applied to contemporary lightweight architectures produce high-quality AI solutions which are both user-accessible and scalable for generating structured queries.

References

[1] Kumar, Rahul, Amar Raja Dibbu, Shrutendra Harsola, Vignesh Subrahmaniam, and Ashutosh Modi. "BookSQL: A Large Scale Text-to-SQL Dataset for Accounting Domain." arXiv preprint arXiv:2406.07860 (2024).

[2] Kang, Hyeongwon, and Pilsung Kang. "Transformer-based multivariate time series anomaly detection using inter-variable attention mechanism." Knowledge-Based Systems 290 (2024): 111507.

[3] Pourreza, Mohammadreza, Ruoxi Sun, Hailong Li, Lesly Miculicich, Tomas Pfister, and Sercan O. Arik. "Sql-gen: Bridging the dialect gap for text-to-sql via synthetic data and model merging." arXiv preprint arXiv:2408.12733 (2024).

[4] Zhang, Bin, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. "Benchmarking the text-to-sql capability of large language models: A comprehensive evaluation." arXiv preprint arXiv:2403.02951 (2024)

[5] Wang, Yusong, Shaoning Li, Tong Wang, Bin Shao, Nanning Zheng, and Tie-Yan Liu. "Geometric transformer with interatomic positional encoding." Advances in Neural Information Processing Systems 36 (2023): 55981-55994.

[6] Dotan, Edo, Gal Jaschek, Tal Pupko, and Yonatan Belinkov. "Effect of tokenization on transformers for biological sequences." Bioinformatics 40, no. 4 (2024): btae196.

[7] Li, Chunhui, Yifan Wang, Zhen Wu, Zhen Yu, Fei Zhao, Shujian Huang, and Xinyu Dai. "Multisql: A schema-integrated context-dependent text2sql dataset with diverse sql operations." In Findings of the Association for Computational Linguistics ACL 2024, pp. 13857-13867. 2024

[8] Meng, Tao, Fuchen Zhang, Yuntao Shou, Hongen Shao, Wei Ai, and Keqin Li. "Masked graph learning with recurrent alignment for multimodal emotion recognition in conversation." IEEE/ACM Transactions on Audio, Speech, and Language Processing (2024)

[9] Li, Jiayao, Li Li, Ruizhi Sun, Gang Yuan, Shufan Wang, and Shulin Sun. "MMAN-M2: Multiple multi-head attentions network based on encoder with missing modalities." Pattern Recognition Letters 177 (2024): 110-120.

[10] Hronsky, Rastislav, and Emmanuel Keuleers. "Tokenization via Language Modeling: the Role of Preceding Text." In Proceedings of the Second Workshop on Computation and Written Language (CAWL)@ LREC-COLING 2024, pp. 23-35. 2024.

[11] Hammad, Ahmed, Stefano Moretti, and M. Nojiri. "Multi-scale cross-attention transformer encoder for event classification." Journal of High Energy Physics 2024, no. 3 (2024): 1-31

[12] Ryu, Jaehee, Seonhee Cho, Gyubok Lee, and Edward Choi. "EHR-SeqSQL: A Sequential Text-to-SQL Dataset For Interactively Exploring Electronic Health Records." arXiv preprint arXiv:2406.00019 (2024).

[13] Chen, Xi, Jinguo You, Likun Likun, and Xiang Li. "Beyond Read-Only: Crafting a Comprehensive Chinese Text-to-SQL Dataset for Database Manipulation and Query." In Findings of the Association for Computational Linguistics: NAACL 2024, pp. 3383-3393. 2024.

[14] Kanburoglu, Ali Bugra, and Faik Boray Tek. "TURSpider: A Turkish Text-to-SQL Dataset and LLM-Based Study." IEEE Access (2024).

[15] Weng, Yuetian, Mingfei Han, Haoyu He, Xiaojun Chang, and Bohan Zhuang. "Longvlm: Efficient long video understanding via large language models." arXiv preprint arXiv:2404.03384 (2024).

[16] Chen, Xiaojun, Tianle Wang, Tianhao Qiu, Jianbin Qin, and Min Yang. "Open-sql framework: Enhancing text-to-sql on open-source large language models." arXiv preprint arXiv:2405.06674 (2024).

[17] Gao, Dawei, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. "Text-to-sql empowered by large language models: A benchmark evaluation." arXiv preprint arXiv:2308.15363 (2023).